SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 0601 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning ***(BL 5)***

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:25**

Annexure A

Coding Challenge

Code of Conduct:

I N.HarshaVardhan (192324189) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **N.HarshaVardhan**

Annexure A

Short Answer Test

1, Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

Program:

```
import math
from collections import Counter

train_data = [
    [160, 55, 'A'],
    [165, 60, 'A'],
    [170, 65, 'A'],
    [175, 70, 'B'],
    [180, 75, 'B'],
    [185, 80, 'B']
]

test_data = [
    [162, 58, 'A'],
    [178, 72, 'B'],
    [168, 63, 'A'],
    [182, 78, 'B']
]

def euclidean_distance(p1, p2):
    return math.sqrt(sum((p1[i] - p2[i]) ** 2 for i in range(len(p1) - 1)))

def knn(train, test, k):
    distances = [(euclidean_distance(test, row), row[-1]) for row in train]
    distances.sort()
    neighbors = [label for _, label in distances[:k]]
    return Counter(neighbors).most_common(1)[0][0]

k = int(input("Enter the value of K: "))

correct = 0

print("\nActual\tPredicted")

for test in test_data:
    predicted = knn(train_data, test, k)
    actual = test[-1]
    print(actual, "\t", predicted)
```

```
    if actual == predicted:
        correct += 1

accuracy = (correct / len(test_data)) * 100

print("\nAccuracy: {:.2f}%".format(accuracy))
```

Output:

```
...  Enter the value of K:  3

     Actual  Predicted
     A        A
     B        B
     A        A
     B        B

     Accuracy: 100.00%
```

2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

Program:

```
import numpy as np

x = np.array([1, 2, 3, 4, 5, 6])
y = np.array([2, 4, 5, 4, 5, 7])

query = float(input("Enter query point: "))
tau = float(input("Enter bandwidth: "))

def locally_weighted_regression(x, y, query, tau):
    weights = np.exp(-((x - query) ** 2) / (2 * tau ** 2))
    X = np.vstack((np.ones(len(x)), x)).T
    W = np.diag(weights)
    theta = np.linalg.pinv(X.T @ W @ X) @ X.T @ W @ y
    return theta[0] + theta[1] * query

def standard_linear_regression(x, y, query):
```

```
    X = np.vstack((np.ones(len(x)), x)).T
    theta = np.linalg.pinv(X.T @ X) @ X.T @ y
    return theta[0] + theta[1] * query

weights = np.exp(-((x - query) ** 2) / (2 * tau ** 2))
lwr_prediction = locally_weighted_regression(x, y, query, tau)
lr_prediction = standard_linear_regression(x, y, query)

print("\nWeights:")
for i in range(len(x)):
    print("x =", x[i], "weight =", round(weights[i], 4))

print("\nLocally Weighted Regression Prediction:", round(lwr_prediction, 4))
print("Standard Linear Regression Prediction:", round(lr_prediction, 4))
```

Output:

```
Enter query point: 3.5
Enter bandwidth: 1

Weights:
x = 1 weight = 0.0439
x = 2 weight = 0.3247
x = 3 weight = 0.8825
x = 4 weight = 0.8825
x = 5 weight = 0.3247
x = 6 weight = 0.0439

Locally Weighted Regression Prediction: 4.5
Standard Linear Regression Prediction: 4.5
```

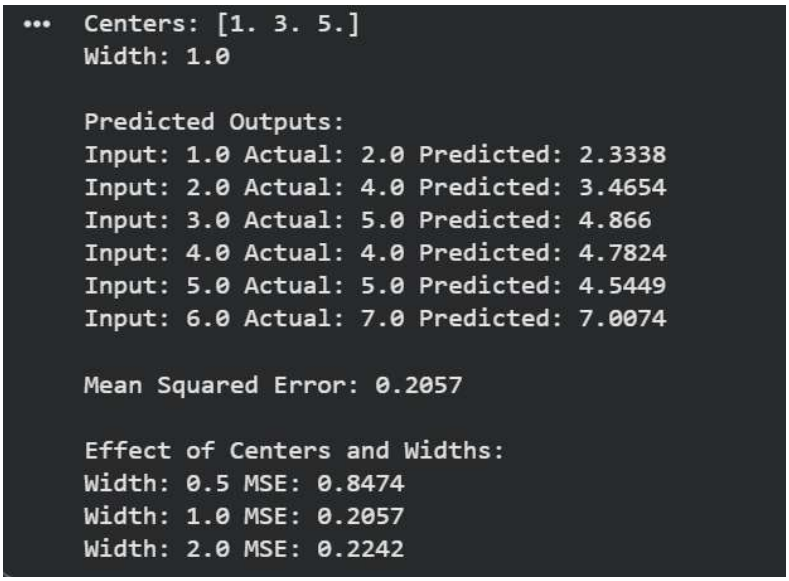
3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.

Program:

```
import numpy as np

x = np.array([1, 2, 3, 4, 5, 6], dtype=float)
y = np.array([2, 4, 5, 4, 5, 7], dtype=float)

centers = np.array([1, 3, 5], dtype=float)
width = 1.0
```

```
def gaussian(x, center, width):
    return np.exp(-((x - center) ** 2) / (2 * width ** 2))

def rbf_features(x, centers, width):
    return np.array([
        [gaussian(value, center, width) for center in centers]
        for value in x
    ])

def train_rbf(x, y, centers, width):
    phi = rbf_features(x, centers, width)
    phi = np.column_stack((np.ones(len(x)), phi))
    weights = np.linalg.pinv(phi) @ y
    return weights

def predict_rbf(x, centers, width, weights):
    phi = rbf_features(x, centers, width)
    phi = np.column_stack((np.ones(len(x)), phi))
    return phi @ weights

weights = train_rbf(x, y, centers, width)
predictions = predict_rbf(x, centers, width, weights)

print("Centers:", centers)
print("Width:", width)
print("\nPredicted Outputs:")
for i in range(len(x)):
    print("Input:", x[i], "Actual:", y[i], "Predicted:", round(predictions[i], 4))

mse = np.mean((y - predictions) ** 2)

print("\nMean Squared Error:", round(mse, 4))

print("\nEffect of Centers and Widths:")

for width_value in [0.5, 1.0, 2.0]:
    weights = train_rbf(x, y, centers, width_value)
    predictions = predict_rbf(x, centers, width_value, weights)
    mse = np.mean((y - predictions) ** 2)
    print("Width:", width_value, "MSE:", round(mse, 4))
```

Output:



4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.

Program:

```
import math

cases = [
    ([25, 40, 1], "Take rest and drink plenty of water"),
    ([30, 80, 2], "Consult a doctor"),
    ([20, 30, 1], "Take rest and stay hydrated"),
    ([35, 90, 3], "Visit a hospital"),
    ([28, 50, 1], "Take rest and monitor symptoms")
]

query = [27, 45, 1]

def euclidean_distance(a, b):
    return math.sqrt(sum((a[i] - b[i]) ** 2 for i in range(len(a))))

distances = []

for features, solution in cases:
    distance = euclidean_distance(query, features)
    distances.append((distance, features, solution))
```

```
distances.sort(key=lambda x: x[0])

print("Query:", query)
print("\nSimilarity Calculation:")
for distance, features, solution in distances:
    print("Case:", features, "Distance:", round(distance, 2))

closest_case = distances[0]

print("\nMost Similar Case:", closest_case[1])
print("Distance:", round(closest_case[0], 2))
print("Solution:", closest_case[2])
```

Output:

```
Query: [27, 45, 1]

Similarity Calculation:
Case: [28, 50, 1] Distance: 5.1
Case: [25, 40, 1] Distance: 5.39
Case: [20, 30, 1] Distance: 16.55
Case: [30, 80, 2] Distance: 35.14
Case: [35, 90, 3] Distance: 45.75

Most Similar Case: [28, 50, 1]
Distance: 5.1
Solution: Take rest and monitor symptoms
```

5. Write a Python program to implement both KNN and RBF models on the same dataset. Compare their predictions for given test inputs. Analyze differences in computation, accuracy, and output behavior, and display results clearly for better comparison.

Program:

```
import numpy as np
from collections import Counter

X = np.array([
    [1, 2],
    [2, 3],
    [3, 3],
    [6, 5],
    [7, 7],
    [8, 6]
```

```
], dtype=float)

y = np.array([0, 0, 0, 1, 1, 1])

test_data = np.array([
    [2, 2],
    [7, 6],
    [4, 4],
    [6, 6]
], dtype=float)

def knn_predict(X, y, test, k):
    distances = np.sqrt(np.sum((X - test) ** 2, axis=1))
    indices = np.argsort(distances)[:k]
    labels = y[indices]
    return Counter(labels).most_common(1)[0][0]

def gaussian(x, center, width):
    return np.exp(-np.sum((x - center) ** 2) / (2 * width ** 2))

def rbf_train(X, y, centers, width):
    phi = np.array([
        [gaussian(x, center, width) for center in centers]
        for x in X
    ])
    phi = np.column_stack((np.ones(len(X)), phi))
    return np.linalg.pinv(phi) @ y

def rbf_predict(test, centers, width, weights):
    phi = np.array([
        [gaussian(test, center, width)]
        for center in centers
    ]).flatten()
    phi = np.insert(phi, 0, 1)
    value = phi @ weights
    return 1 if value >= 0.5 else 0

k = 3
width = 1.5
centers = X[[0, 2, 4]]

weights = rbf_train(X, y, centers, width)

knn_predictions = []
rbf_predictions = []

for test in test_data:
    knn_predictions.append(knn_predict(X, y, test, k))
    rbf_predictions.append(rbf_predict(test, centers, width, weights))
```

```
print("KNN vs RBF Comparison")
print("-" * 45)
print("Test Input\tKNN\tRBF")

for i, test in enumerate(test_data):
    print(
        str(test) + "\t",
        knn_predictions[i],
        "\t",
        rbf_predictions[i]
    )

knn_accuracy = np.mean(
    np.array(knn_predictions) == np.array([0, 1, 0, 1])
) * 100

rbf_accuracy = np.mean(
    np.array(rbf_predictions) == np.array([0, 1, 0, 1])
) * 100

print("\nKNN Accuracy:", round(knn_accuracy, 2), "%")
print("RBF Accuracy:", round(rbf_accuracy, 2), "%")

print("\nAnalysis")
print("KNN uses distances to the K nearest training samples and majority voting.")
print("RBF uses Gaussian activation functions based on distances from selected centers.")
print("KNN has simpler training but performs distance calculations during prediction.")
print("RBF requires training weights but provides smooth, center-based predictions.")
print("The predictions can differ because KNN uses discrete neighbor voting while RBF uses continuous Gaussian responses.")
```

Output:

```
KNN vs RBF Comparison
---------------------------------------------
Test Input      KNN     RBF
[2. 2.]  0       0
[7. 6.]  1       1
[4. 4.]  0       1
[6. 6.]  1       1

KNN Accuracy: 100.0 %
RBF Accuracy: 75.0 %

Analysis
KNN uses distances to the K nearest training samples and majority voting.
RBF uses Gaussian activation functions based on distances from selected centers.
KNN has simpler training but performs distance calculations during prediction.
RBF requires training weights but provides smooth, center-based predictions.
The predictions can differ because KNN uses discrete neighbor voting while RBF uses continuous Gaussian responses.
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**20**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand